

Mohamed El Laithy

Content Creator

Core Architecture Of Spring Boot Microservice Complete Guide

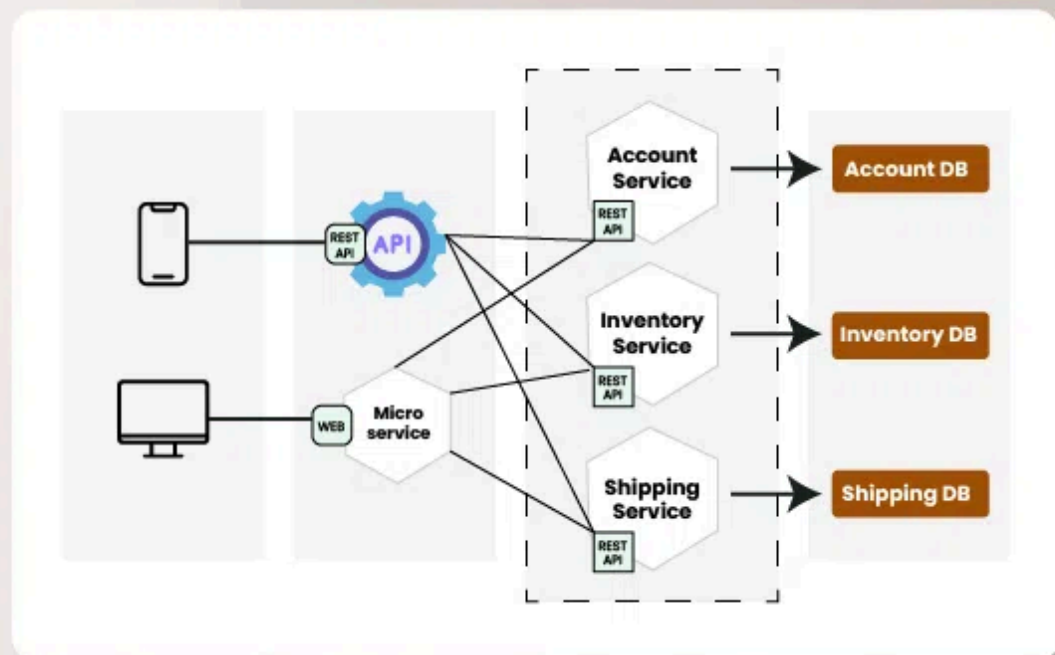
Swipe for more



Overview

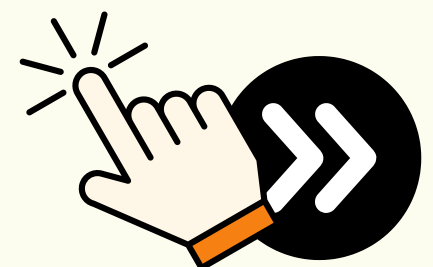
Microservices are an architectural approach to developing software applications as a collection of small, independent services that communicate with each other over a network. Instead of building a monolithic application where all the functionality is tightly integrated into a single codebase, microservices break down the application into smaller, loosely coupled services.

Microservices



- This architecture allow you to take a large monolith application and decompose it into small manageable components/services. Also, it is considered as the building block of modern applications.
- Microservices can be written in a variety of programming languages, and frameworks, and each service acts as a mini-application on its own.

Swipe for more



Core Components of Microservices Architecture

what is Core Components of Microservices Architecture:

1. Service Discovery
2. Load Balancer
3. API Gateway
4. Service Registry
5. Circuit Breaker
6. Monitoring Tools
7. Orchestration Layer
8. Configuration Servers

1- Service Discovery

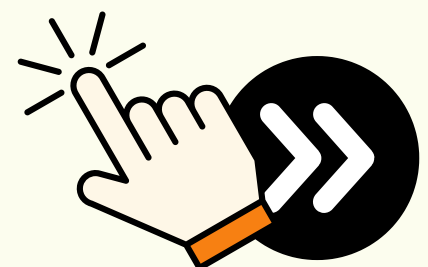
What is Service Discovery?

Service Discovery is a mechanism in microservices architecture that allows services to automatically find and communicate with each other without hard-coding IP addresses or ports.

Why Service Discovery is needed?

- Services scale up/down dynamically
- IPs & ports change frequently
- Manual configuration is not possible

Swipe for more

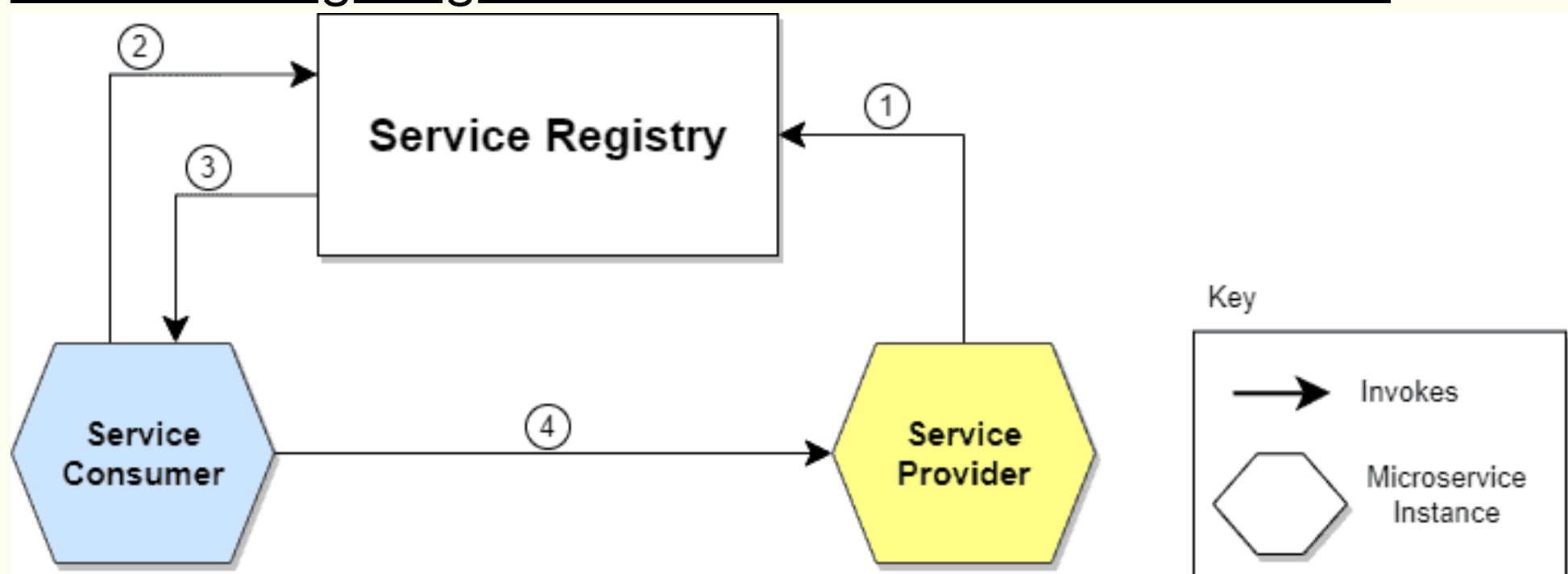


How Does Service Discovery Works?

Service Discovery handles things in two parts. First, it provides a mechanism for an instance to register and say, “I’m here!” Second, it provides a way to find the service once it has registered.

Let’s clarify the concept we’ve discussed so far with an example: a Service Consumer and a Service Provider (a service exposing REST API). The Service Consumer needs the Service Provider to read and write data.

The following diagram shows the communication flow:



Let’s describe the steps illustrated in the diagram:

1. The location of the Service Provider is sent to the Service Registry (a database containing the locations of all available service instances).
2. The Service Consumer asks the Service Discovery Server for the location of the Service Provider.
3. The location of the Service Provider is searched by the Service Registry in its internal database and returned to the Service Consumer.
4. The Service Consumer can now make direct requests to the Service Provider.

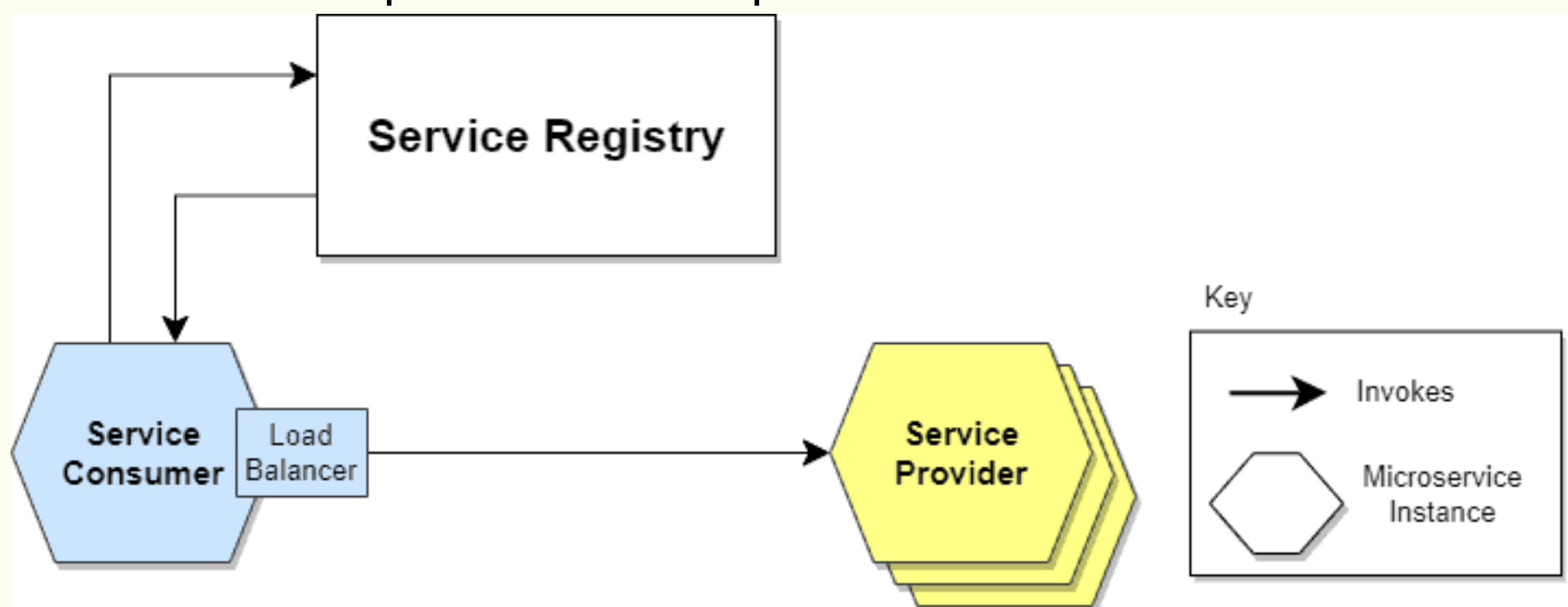
Swipe for more



Service Discovery Implementations

1- Client-Side Service Discovery

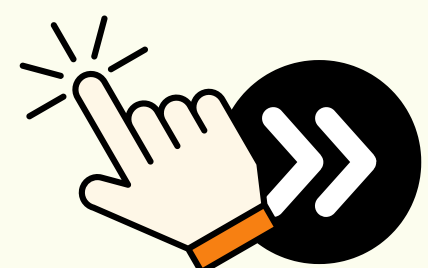
When using Client-Side Discovery, the Service Consumer is responsible for determining the network locations of available service instances and load balancing requests between them. The client queries the Service Registry. Then the client uses a load-balancing algorithm to choose one of the available service instances and performs a request.



Giving responsibility for client-side load balancing is both a burden and an advantage. It's an advantage because it saves an extra hop that we would've had with a dedicated load balancer. It's a disadvantage because the Service Consumer must implement the load balancing logic.

We can also point out that the Service Consumer and the Service Registry are quite coupled. This means that Client-Side Discovery logic must be implemented for each programming language and framework used by the Service Consumers.

Swipe for more

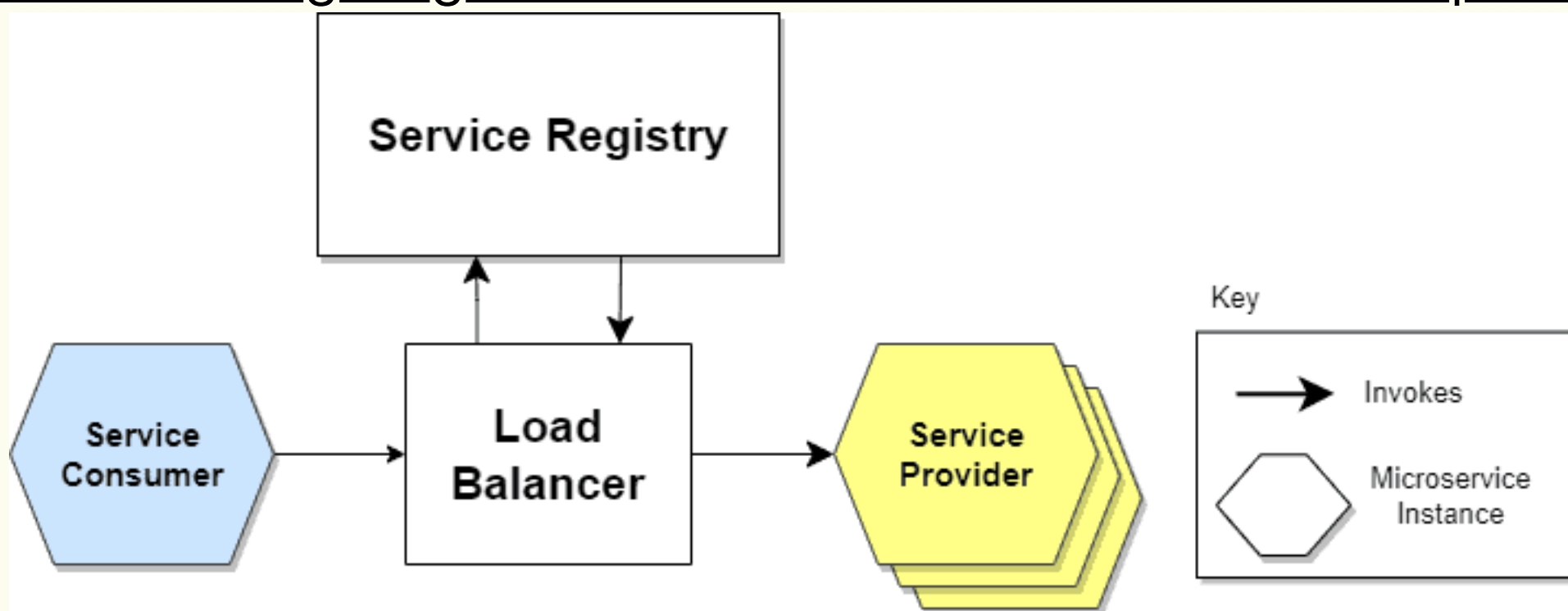


Service Discovery Implementations

2- Server-Side Service Discovery

The alternate approach to Service Discovery is the Server-Side Discovery model, which uses an intermediary that acts as a Load Balancer. The client makes a request to a service via a load balancer that acts as an orchestrator. The load balancer queries the Service Registry and routes each request to an available service instance.

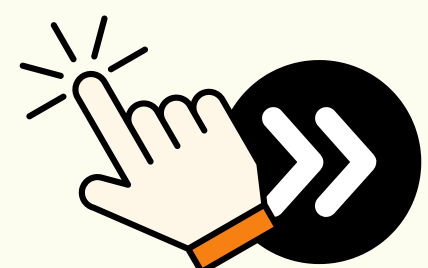
The following diagram shows how communication takes place:



In this approach, a dedicated actor, the Load Balancer, does the job of load balancing. This is the main advantage of this approach. Indeed, creating this level of abstraction makes the Service Consumer lighter, as it doesn't have to deal with the lookup procedure. As a matter of fact, there's no need to implement the discovery logic separately for each language and framework that the Service Consumer uses.

On the other hand, we must set up and manage the Load Balancer, unless it's already provided in the deployment environment.

Swipe for more



2- Load Balancing in Microservices

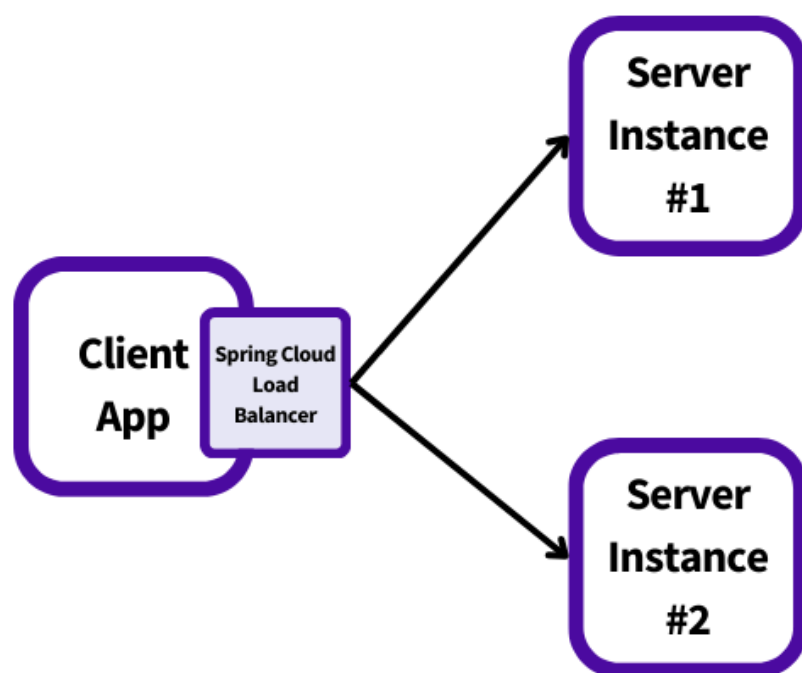
What is Load Balancing?

Load balancing is the process of distributing incoming network traffic across multiple servers or resources to ensure optimal utilization and prevent overload on any single server. This helps improve the scalability, reliability, and performance of applications.

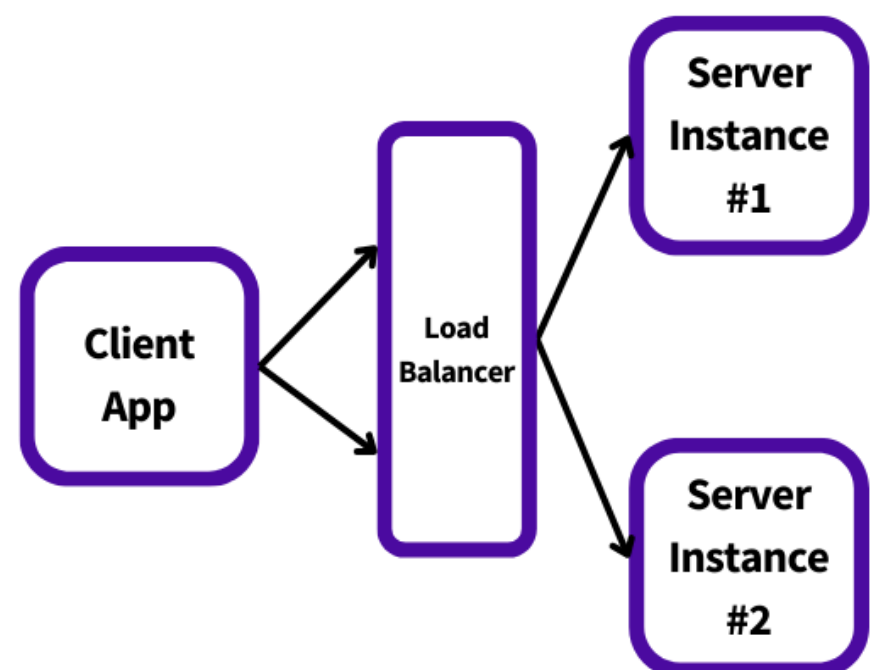
Why is Load Balancing Important?

- **Scalability:** It allows applications to handle increased traffic by distributing it evenly across multiple servers.
- **High Availability:** Load balancers ensure that if one server fails, the traffic can be redirected to other healthy servers, minimizing downtime.
- **Optimized Performance:** By distributing traffic based on various algorithms, load balancers optimize the performance of the application.

Types of Load Balancing



Client Side Load
Balancing



Server Side Load
Balancing

Swipe for more

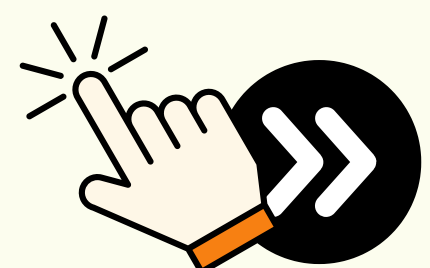


Understanding Load Balancing Algorithms

Load balancing algorithms play a crucial role in efficiently distributing incoming requests across servers, ensuring optimal performance and resource utilization. Let's dive into a brief overview of these load balancing algorithms.

- **Round Robin:** Requests are distributed across servers in a circular order. Each server takes its turn handling requests, ensuring a fair distribution. However, it doesn't consider each server's current workload.
- **Least Connections:** Requests are sent to the server with the fewest active connections. This aims to balance the load evenly among servers. Yet, it may overlook the complexity or duration of each request.
- **Least Time:** Requests are directed to the server with the fastest response time and the fewest active connections. This algorithm prioritizes efficiency and responsiveness, exclusive to certain load balancers.
- **Hash:** Requests are distributed based on a unique code derived from the request, like the client's IP address or the request's URL. This ensures consistency, useful for maintaining session persistence or caching.
- **Random:** Requests are randomly assigned to available servers. While simple, this method may lead to uneven distribution in certain scenarios.
- **Random with Two Choices:** Requests are randomly assigned to two servers, and then the less loaded one is selected. This reduces the risk of overloading a single server, improving system reliability.

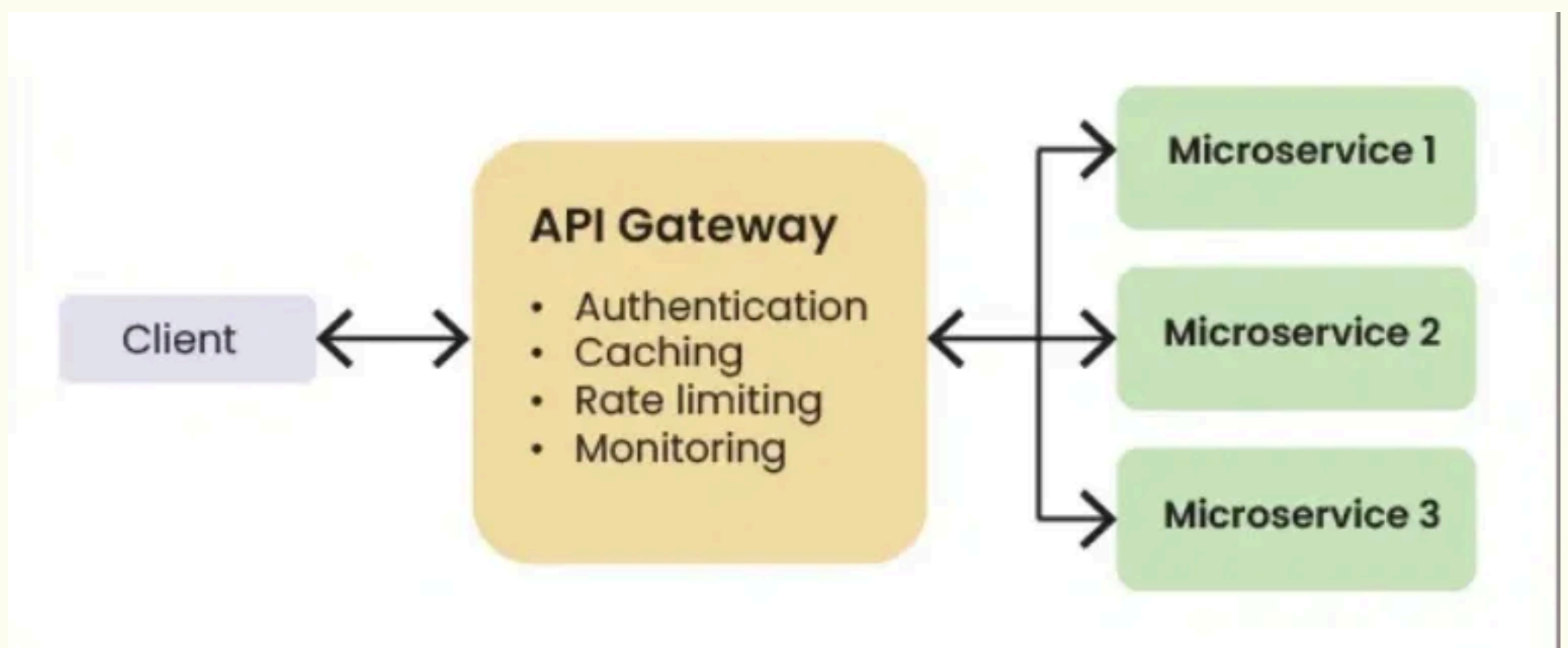
Swipe for more



3- API Gateway

What is API Gateway?

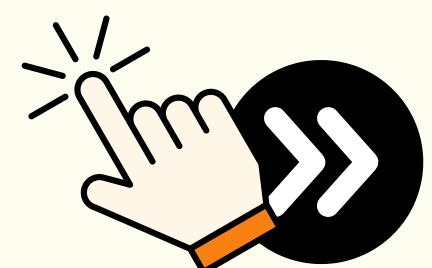
An API Gateway is a key component in system design, particularly in microservices architectures and modern web applications. It serves as a centralized entry point for managing and routing requests from clients to the appropriate microservices or backend services within a system.



Benefits of using an API Gateway

- **Centralized Entry Point** – Clients interact with a single gateway to access multiple microservices.
- **Routing & Load Balancing** – Directs requests to the appropriate service and distributes load across instances.
- **Authentication & Authorization** – Verifies identity and enforces access control using JWTs, OAuth, or API keys.
- **Request & Response Transformation** – Converts data formats (e.g., JSON ↔ XML) for compatibility between services.

Swipe for more

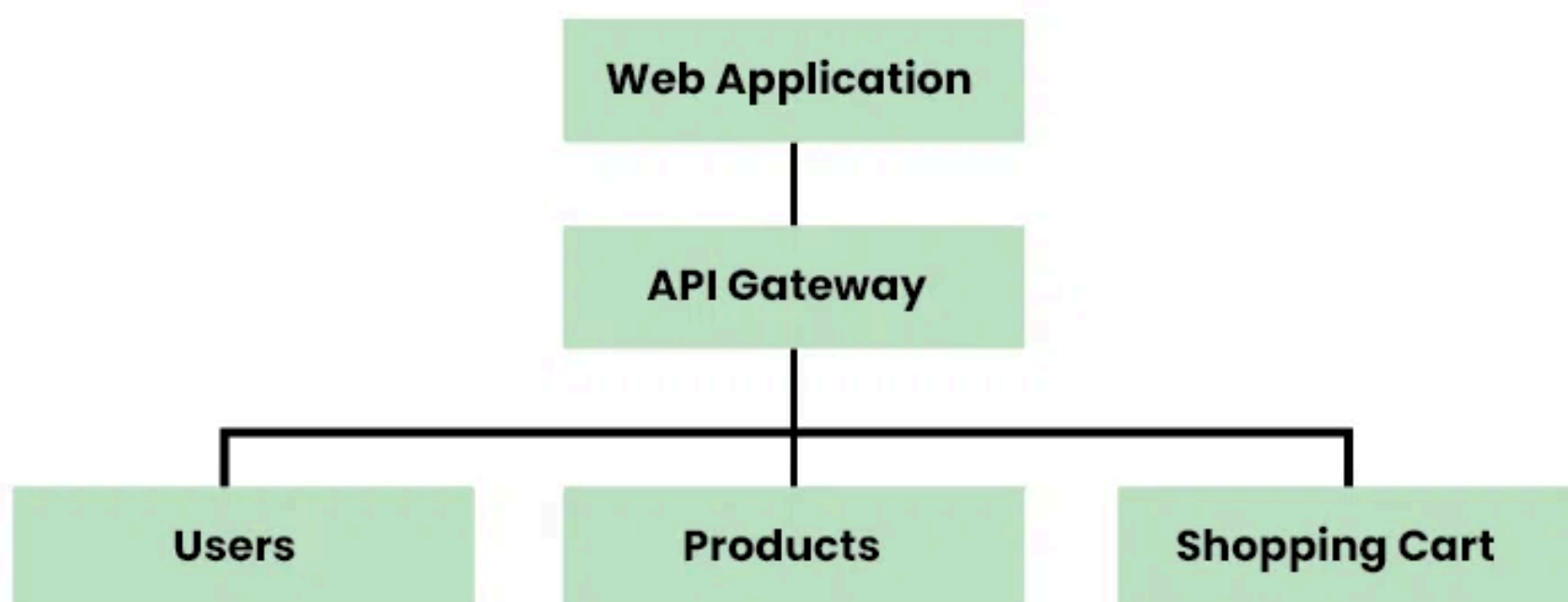


How does API Gateway work?

Let us see how API Gateway works:

- Routing – Directs client requests to the appropriate service based on URL, method, or headers.
- Protocol Translation – Converts requests between protocols (e.g., HTTP → gRPC/WebSocket).
- Request Aggregation – Combines multiple backend calls into one to reduce round trips.
- Authentication & Authorization – Verifies client identity and access permissions.
- Rate Limiting & Throttling – Controls request rates to prevent abuse and ensure resource balance.
- Load Balancing – Distributes requests across service instances for scalability and availability.
- Caching – Stores backend responses to speed up repeated requests.
- Monitoring & Logging – Tracks metrics and logs for performance and usage insights.

API Gateway with Microservices Example



API Gateway with Microservices

Swipe for more



4- Service Registry

What is Service Registry?

A Service Registry serves as a centralized database or directory where information about available services and their locations is stored and maintained. It acts as a vital component of service discovery by providing a central point for service registration, lookup, and management.

Below is how a Service Registry typically works:

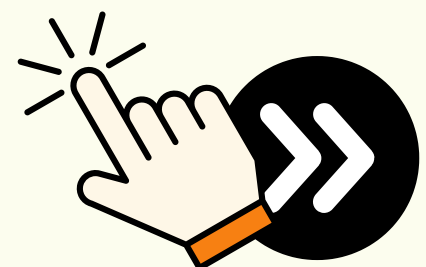
1. Service Registration

When a microservice instance starts up or becomes available, it registers itself with the Service Registry. This registration process includes providing metadata such as the service name, network location (IP address and port), health status, and possibly other attributes.

2. Service Lookup

Clients or other microservices that need to communicate with a particular service do not have prior knowledge of the service's location. Instead, they query the Service Registry to dynamically discover the available instances of the desired service. The Service Registry responds with the network location of one or more service instances.

Swipe for more



3. Health Monitoring

Service Registries often include health-checking mechanisms to monitor the status of registered service instances. This allows the registry to detect and remove unhealthy or unavailable instances from its database, ensuring that clients only receive information about healthy and operational services.

4. Load Balancing

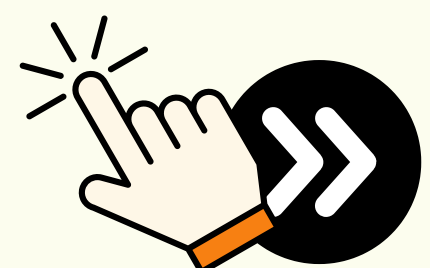
Some Service Registries incorporate load-balancing capabilities to distribute incoming requests among multiple instances of the same service. This helps in optimizing resource utilization, improving performance, and enhancing fault tolerance.

5. Dynamic Updates

Service instances may come and go dynamically due to scaling, deployment changes, or failures. The Service Registry continuously updates its records to reflect changes in the availability and health status of services, ensuring that clients receive up-to-date information.

Overall, a Service Registry serves as a central hub for managing service discovery and communication in a microservices architecture. It provides a scalable and resilient solution for dynamically locating and accessing services within a distributed system, promoting agility, scalability, and reliability.

Swipe for more



5- Circuit Breaker

What is Circuit Breaker?

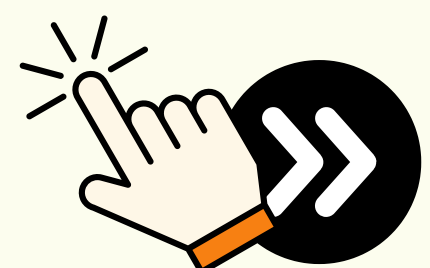
The Circuit Breaker pattern is like a safety switch for microservices. Suppose your online store depends on a payment service. If that service starts failing repeatedly, the circuit breaker “trips” and stops further calls for a while, preventing cascades and giving the service time to recover. Let's understand circuit breaker pattern with this example:

- Your store makes a request to the payment service to process a payment. Everything works fine.
- Suddenly, the payment service has issues and fails three times in a row.
- The circuit breaker trips and enters an "open" state. Now, when your store tries to contact the payment service, it immediately gets an error response instead of trying to connect again.
- After a set time, the circuit breaker changes to a "half-open" state. It allows a few test requests to see if the payment service is back online.
- If those requests succeed, the circuit breaker resets to "closed," and everything goes back to normal. If they fail, it stays open longer, giving the payment service more time to recover.

Characteristics of Circuit Breaker Pattern

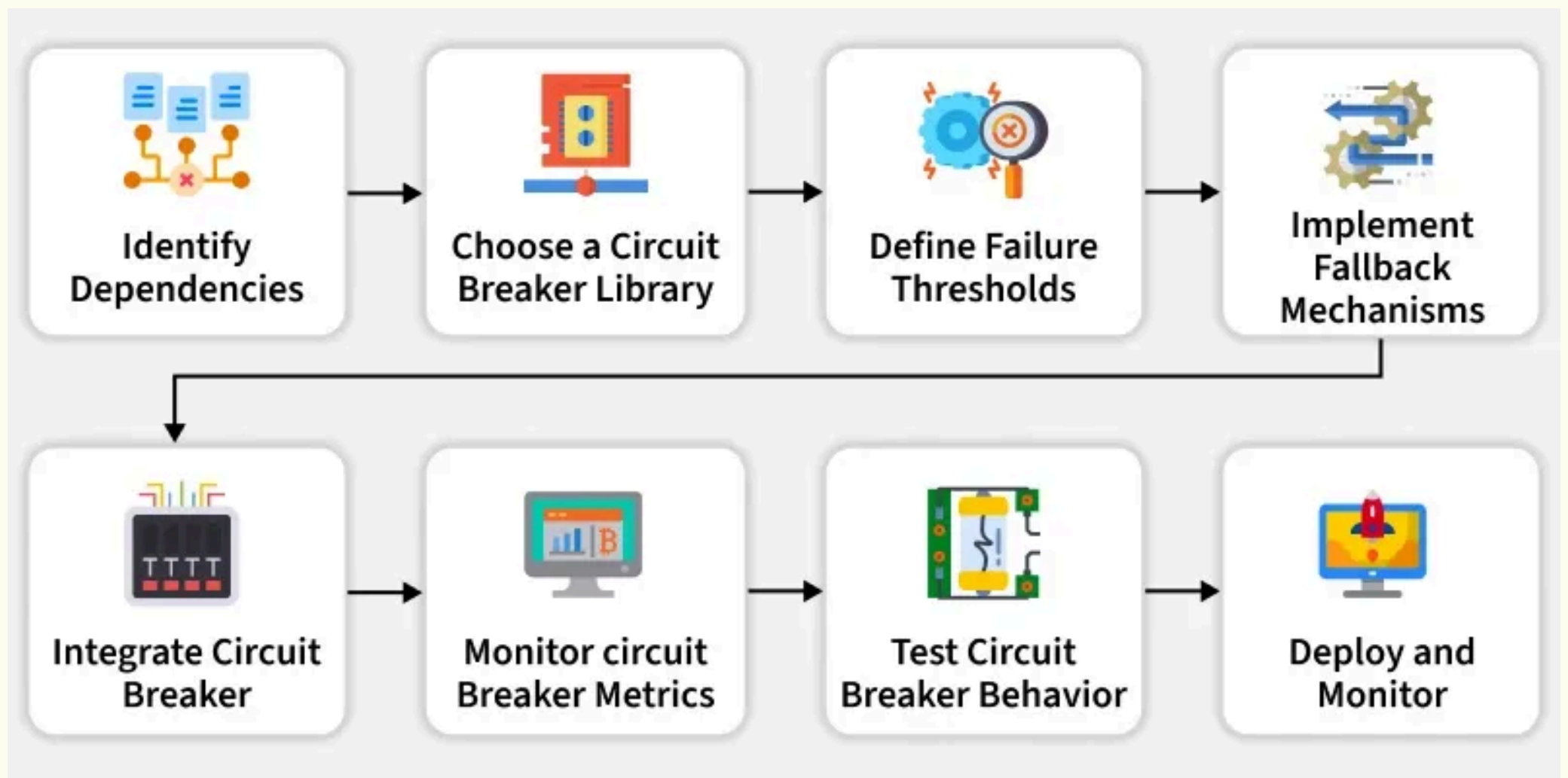
- Improves fault tolerance by isolating failing services.
- Monitors latency, error rate, and timeouts.
- Prevents cascading failures.
- Supports fallbacks (cached data, default responses, queues).
- Auto-recovers when the service stabilizes.

Swipe for more



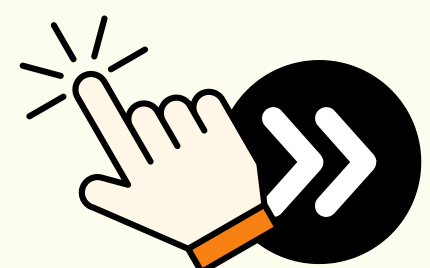
Steps to Implement Circuit Breaker Pattern

Below are the steps to implement Circuit Breaker Pattern:



- Step 1: Identify Dependencies: Going for the external services that will bring interactions and make the microservice functional in turn.
- Step 2: Choose a Circuit Breaker Library: Choose a circuit breaker library from these existing libraries, or create your own framework that you are familiar with, based on your programming language and platform.
- Step 3: Integrate Circuit Breaker into Code: Make sure to insert the selected circuit breaker library into your microservices code base.
- Step 4: Define Failure Thresholds: Set boundaries for faults and time-outs that turn the mechanism of the circuit breaker to open.

Swipe for more



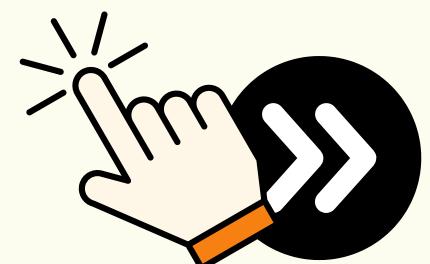
- Step 5: Implement Fallback Mechanisms: Include whenever the circuit has open or close requests, the fallback mechanism should be implemented.
- Step 6: Monitor Circuit Breaker Metrics: Use the statistics built into the circuit breaker library to see the health and behaviour of your services. Such evaluation measurements includes number of successful/unsuccessful requests, status of the circuit breaker, and error rates.
- Step 7: Tune Configuration Parameters: Tuning configuration parameters like timeouts, thresholds, and retry methods in accordance to behavior of your microservices and your application requirements.
- Step 8: Test Circuit Breaker Behavior: Perform live testing of your circuit breaker during different operating states including normal function, failure scenarios (high load), and fault condition.
- Step 9: Deploy and Monitor: Move/deploy your microservice with circuit breaker, into your production environment.

Use Cases of Circuit Breaker Pattern

Below are the use cases of circuit breaker pattern:

- When microservices are communicating with one over the network through the pattern of the Circuit breaker, the document helps to deal with the network failures and with unavailability of service or with slow responses.
- That, in doing so, it avoids collateral damage of failures by serving as a barrier between a failed service and providing alternative options when failure occurs.
- That is, the microservices are the APIs or services which may be external or from other parties.
- This Circuit Breaker pattern can be included as a contingency to mitigate against failures in the integrations, enabling the whole system to stay functional even when the external parties are affected by unforeseen issues.

Swipe for more



6- Monitoring Tools in Microservices

Monitoring is essential in microservices because multiple services run independently. Proper monitoring helps detect issues early, ensure high availability, and improve performance.

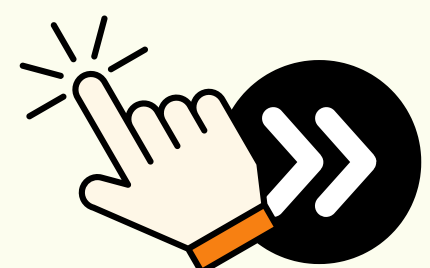
1. Prometheus

- Type: Open-source metrics collection and monitoring tool.
- Features:
 - Collects metrics from microservices using HTTP endpoints.
 - Stores metrics in a time-series database.
 - Works well with Grafana for visualization.
- Use Case: Track CPU, memory, request latency, error rates.

2. Grafana

- Type: Open-source dashboard and visualization tool.
- Features:
 - Visualizes metrics from Prometheus, InfluxDB, Elasticsearch, etc.
 - Create dashboards for system health, request latency, throughput.
- Use Case: Real-time dashboards for microservices performance.

Swipe for more



3. ELK Stack (Elasticsearch, Logstash, Kibana)

- Type: Centralized logging solution.
- Features:
 - Collects logs from all services (Logstash).
 - Indexes and stores logs (Elasticsearch).
 - Visualizes logs and trends (Kibana).
- Use Case: Debugging errors, analyzing request patterns.

4. Zipkin / Jaeger

- Type: Distributed tracing tools.
- Features:
 - Trace requests as they flow across multiple services.
 - Identify bottlenecks, latency, and slow services.
- Use Case: Visualize request paths in Spring Boot microservices.

5. Micrometer

- Type: Metrics instrumentation library for Spring Boot.
- Features:
 - Integrates with Prometheus, Grafana, Datadog.
 - Collects application-level metrics: request counts, latency, errors.
- Use Case: Expose Spring Boot service metrics automatically.

6. New Relic / Datadog / AppDynamics

- Type: Commercial APM (Application Performance Monitoring) tools.
- Features:
 - End-to-end monitoring of microservices.
 - Detect anomalies, bottlenecks, and errors automatically.
- Use Case: Large-scale enterprise microservices monitoring.

Swipe for more



7- Microservices Orchestration Layer

What is Microservices Orchestration

Microservices orchestration is the deployment and coordination of a network of microservices within a distributed system. It is a layer over them that ensures they work together nicely in data workflows. Microservices orchestration handles a few key tasks:

- Ensuring microservices run when they're supposed to, and in the correct order
- Retrying failed microservices immediately or rolling back to a clean slate (without affecting the rest of the pipeline)
- Scaling each microservice to handle its workload
- Adding observability to microservices to debug issues

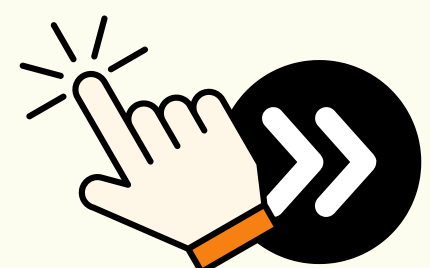
Without microservice orchestration tools automating these steps, data teams would be overwhelmed to do each of these steps for each of their many data workflows.

Why You Need Microservices Orchestration

1. Controlled Execution
2. Resilient & Debuggable Workflows
3. Automatic Scaling & Resource Management

Orchestration prevents execution chaos, improves reliability, simplifies debugging, and ensures efficient scaling in microservices architectures.

Swipe for more



When to Use Microservice Orchestration

Ideal for workflows that:

- Have many subtasks (ingestion, processing, storage, notification).
- Use custom infrastructure (databases, servers, pipelines).
- Are event-triggered, needing rapid handling of spikes or failures.
- Are exposed via APIs for internal or external use.

Common Use Cases (E-commerce Examples)

1. Data Analytics

- Workflow: Ingest transactions → Process monthly metrics → Store results → Generate reports.
- Orchestration ensures correct execution order, retries failing components, and allows adding new microservices seamlessly.

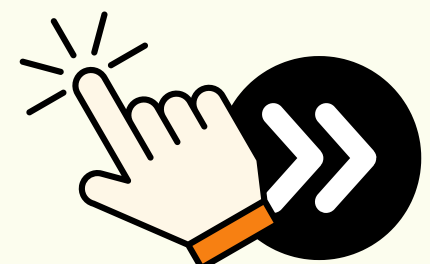
2. Machine Learning

- Workflow: Data collection → Cleaning → Feature selection → Model training → Inference → Logging.
- Orchestrators manage complex ML pipelines, ensure proper sequencing, handle GPU/resource scaling, and manage high-inference loads.

3. AI Agents

- Workflow: Multiple agents performing independent tasks (top results, info gathering, shipping optimization).
- Orchestrators manage sequencing, data flow, resource allocation, retries, and performance monitoring for hundreds of agent calls.

Swipe for more



8- Configuration Servers

What is Microservices Config Server

In a microservices architecture, each service may need different configurations (databases, API URLs, credentials, feature flags).

Managing these individually becomes error-prone and hard to maintain. A Config Server centralizes configuration management and provides it to all microservices dynamically.

Key Roles of a Config Server

1. Centralized Configuration Management
 - Stores all microservice configurations in a single repository (Git, SVN, filesystem).
 - Avoids duplicating config in each service.
2. Environment-Specific Configurations
 - Supports dev, test, prod environments.
 - Services fetch the correct config based on their profile.
3. Dynamic Refresh / Hot Reload
 - Microservices can reload configuration at runtime without restarting.
 - Useful for feature flags or endpoint changes.
4. Security & Version Control
 - Sensitive data can be encrypted.
 - Config changes are version-controlled in Git.
5. Consistency Across Services
 - Ensures that all services use the same configuration values.
 - Prevents mismatched or outdated settings.

Swipe for more



If you
find this
helpful, please
like and share
it with your
friends



Created By:
Mohamed El Laithy.

